

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 05: Quantization and Low-Rank Adaptation

Complete notebook-aligned solution

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 Quantization **2**
1.1 Absmax quantization and accumulation 2
1.2 Min-max quantization and the zero-point correction 4
1.3 Dynamic quantization on a decoder-only model 6

2 LoRA **11**
2.1 Build and train the manual adapter 11
2.2 Apply the same encoder adapters with PEFT 15

A Diagnostic output and earlier authentication attempt **19**

B Execution and reproducibility notes **19**

1 Quantization: from low-bit arithmetic to Llama inference

This exercise first makes low-bit arithmetic visible on a small matrix and vector, where symmetric and asymmetric quantization can be compared directly. It then applies post-training quantization to Llama 3.2 1B, measuring serialized size and a fixed CPU generation workload across float32, PyTorch int8, and BitsAndBytes int4. The toy example isolates rounding, zero points, and accumulation overflow; the model experiment shows how these ideas behave in a real decoder-only network.

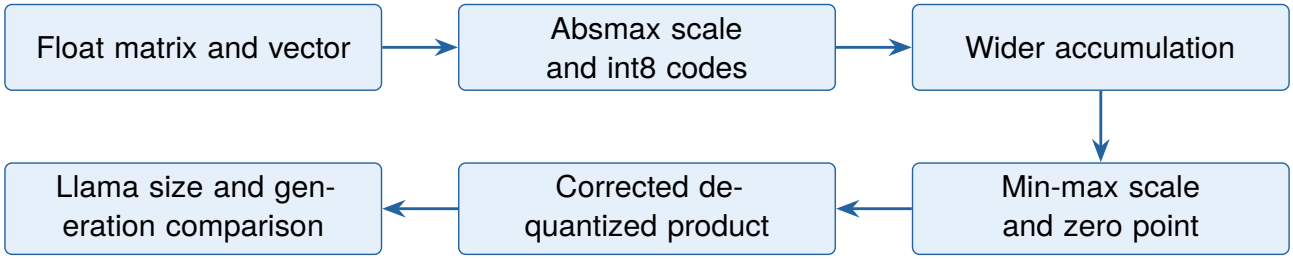


Figure 1: The exercise pipeline, from input to checked result.

1.1 Absmax quantization and accumulation

We start with a fixed seed and two small tensors: a 3 by 5 weight matrix and a length-five input vector. Setting one coordinate in each to zero makes it possible to observe how the two quantizers treat zero.

```
import torch

torch.random.manual_seed(0)

n_rows = 3
n_cols = 5

# TODO: Create two random tensors, W (of shape n_rows, n_cols) and x (of shape n_cols)
W = torch.randn(n_rows, n_cols)
x = torch.randn(n_cols)

W[0,0] = 0
x[0] = 0
```

These tensors are the common reference for the unquantized product and both quantization schemes. Their small dimensions keep the integer arithmetic inspectable.

Before changing precision, this product records the reference output of the original matrix and vector. Every later reconstruction is compared with this value.

```
out = W @ x
print(out)
```

```
tensor([0.4829, 1.3345, 0.2121])
```

The floating-point result is the numerical target for the quantized calculations that follow.

This helper implements symmetric absmax quantization. Its scale maps the largest magnitude to 127, rounding maps values to integer levels, and clamping keeps them in the symmetric range from -127 to 127. The zero-range guard makes an all-zero tensor safe to quantize.

```
def absmax_quantize(W):
    # NOTE: we assume that we always map to 8-bit integers

    # TODO: find the scale factor that maps the maximum absolute value in W to the maximum value of int8
    max_value = W.abs().max()
    scale = torch.where(max_value > 0, max_value / 127, torch.ones_like(max_value)) # how "long" the step between any two
     int8 values is

    # TODO: quantize W using the scale factor (hint: remember to round to the nearest integer and convert to int8)
    W_q = torch.round(W / scale).clamp(-127, 127).to(torch.int8)
    return W_q, scale
```

The function returns both integer codes and the scale needed to recover approximate floating-point values.

Quantization is useful only if the integer representation can be interpreted again. This inverse helper converts the codes to float and multiplies by their saved scale.

```python
def absmax_dequantize(W_q, scale):
    # TODO: dequantize W_q using the scale factor
    dequantized_W = W_q.to(torch.float32) * scale
    return dequantized_W
```

The original scale is required downstream; without it the integer codes do not retain the magnitude of the source tensor.

We now apply the same symmetric scheme independently to the matrix and vector. Independent scales use each operand's available integer range effectively.

```python
W_q, scale_W = absmax_quantize(W)
x_q, scale_x = absmax_quantize(x)
```

The resulting integer tensors and separate scales feed the reconstruction checks and integer dot product.

This comparison examines the matrix codes, the original weights, and the dequantized approximation. The mean absolute difference condenses coordinate-wise rounding error into one diagnostic.

```python
print(W_q)
print(W)

#TODO: dequantize W_q and check how close it is to the original W
W_deq = absmax_dequantize(W_q, scale_W)
print(W_deq)
W_diff = (W - W_deq).abs().mean()
print(W_diff)
```

```
tensor([[   0,  -17, -127,   33,  -63],
        [ -82,   24,   49,  -42,  -24],
        [ -35,   11,  -50,   64,  -62]], dtype=torch.int8)
tensor([[ 0.0000, -0.2934, -2.1788,  0.5684, -1.0845],
        [-1.3986,  0.4033,  0.8380, -0.7193, -0.4033],
        [-0.5966,  0.1820, -0.8567,  1.1006, -1.0712]])
tensor([[ 0.0000, -0.2916, -2.1788,  0.5661, -1.0808],
        [-1.4068,  0.4117,  0.8406, -0.7205, -0.4117],
        [-0.6005,  0.1887, -0.8578,  1.0980, -1.0637]])
tensor(0.0039)
```

The reconstructed matrix stays close to the original and preserves the deliberately zero entry. The small nonzero mean error is quantization loss, not a change to the original tensor.

The same round trip is checked for the input vector, using its own scale. This separates weight error from activation error.

```python
print(x_q)
print(x)

# TODO: dequantize x_q and check how close it is to the original x
x_deq = absmax_dequantize(x_q, scale_x)
print(x_deq)
x_diff = (x - x_deq).abs().mean()
print(x_diff)
```

```
tensor([   0,  -48,   31,  -75, -127], dtype=torch.int8)
tensor([ 0.0000, -0.5663,  0.3731, -0.8920, -1.5091])
tensor([ 0.0000, -0.5704,  0.3684, -0.8912, -1.5091])
tensor(0.0019)
```

The vector reconstruction also preserves zero; its mean absolute error is reported independently from the matrix's.

This is the tempting low-precision computation: multiply the quantized matrix and vector while they are still int8. It exposes a hardware arithmetic issue separate from the rounding errors measured above.

```python
# TODO: perform the matrix multiplication using the quantized values
product = W_q @ x_q
print(product)
```

```
tensor([101, -91, -28], dtype=torch.int8)
```

The displayed result has wrapped values because the dot products exceed the int8 range. Quantized operands therefore need a wider accumulator even when their storage uses eight bits.

Inspecting the first matrix row and the vector lets us verify one dot product by hand and compare it with the wrapped result.

```
W_q[0], x_q
```

```
(tensor([   0,  -17, -127,   33,  -63], dtype=torch.int8),
 tensor([   0,  -48,   31,  -75, -127], dtype=torch.int8))
```

The integer operands are individually representable in int8; their products' sum is not. This isolates accumulation overflow as the cause.

The corrected calculation promotes both operands to int16 before multiplication and accumulation. This is wide enough for these short illustrative rows and prevents int8 wraparound.

```
# TODO: perform the matrix multiplication using the quantized values converted to int16
out_q = W_q.to(torch.int16) @ x_q.to(torch.int16)
print(out_q)
```

```
tensor([2405, 6565,  996], dtype=torch.int16)
```

The larger integer values are the proper dot products in quantized-code space. Production kernels generally accumulate low-bit products into a wider type as well.

The integer dot products are converted back to the original output scale by multiplying the matrix and vector scales. This follows from independently scaled operands: each product carries both factors.

```
# TODO: dequantize the result and check how close it is to the original result
out_deq = out_q.to(torch.float32) * scale_W * scale_x

print(out_deq)
print(out)
```

```
tensor([0.4903, 1.3383, 0.2030])
tensor([0.4829, 1.3345, 0.2121])
```

The reconstructed output is close to the original floating-point product; the remaining difference is the accumulated effect of quantizing both operands.

A final scalar error summarizes the effect of absmax quantization on the matrix-vector result, rather than on either input alone.

```
#TODO: check the difference between the original and the dequantized result
out_diff = (out - out_deq).abs().mean()
print("Mean absolute output error:", out_diff.item())
```

```
Mean absolute output error: 0.006753509398549795
```

The measured mean absolute output error is about 0.00675 for this seeded example. It is an observed result for these tensors, not a general error bound.

1.2 Min-max quantization and the zero-point correction

Symmetric absmax mapping dedicates its range around zero, so it cannot use the extra negative int8 endpoint. Min-max mapping instead maps observed extrema to -128 and 127. With zero points z_W and z_x, the recovered product must expand $(q_W - z_W)(q_x - z_x)$: subtract $z_x \sum q_W$ and $z_W \sum q_x$, then add $K z_W z_x$, where K is the inner dimension. Only after this correction do we divide by both scales.

The second helper uses an asymmetric min-max mapping. It stretches the observed range to the full int8 interval, stores a zero point, and reverses both operations during dequantization. The constant-range guard avoids division by zero.

```
def minmax_quantize(W):
    # the following notations come from:
    # (1) scaling W to [0,1] ==> W' = (W - min(W)) / (max(W) - min(W)),
    # (2) scaling W' to [-128, 127] ==> W_q = W' * 255 - 128
    # by combining the two, we get that:
    # W_q = W * scale + offset
    # we will call the offset "zero_point", as it represent the value that maps to 0

    # TODO: find the scale factor that maps the minimum and maximum values in W to -128 and 127
    delta = W.max() - W.min() # the range of values in W
    scale = torch.where(delta > 0, 255 / delta, torch.ones_like(delta))
    zero_point = -128 - W.min() * scale # the value that maps to 0

    # TODO: quantize W using the scale factor (hint: remember to round to the nearest integer and convert to int8)
    W_q = torch.round(W * scale + zero_point).clamp(-128, 127).to(torch.int8)
```

```
    return W_q, scale, zero_point

def minmax_dequantize(W_q, scale, zero_point):
    # TODO: dequantize W_q using the scale factor and zero_point
    dequantized_W = (W_q.to(torch.float32) - zero_point) / scale
    return dequantized_W
```

Unlike absmax, this mapping can use all 256 codes and need not map real zero to integer zero. The returned scale and zero point are both required to interpret the codes.

We quantize the same matrix and vector with independent min-max ranges. Reusing the same examples makes the later error comparison meaningful.

```
W_q, scale_W, zero_point_W = minmax_quantize(W)
x_q, scale_x, zero_point_x = minmax_quantize(x)
```

Each operand now carries its own scale and zero point; these are needed for both reconstruction and the corrected dot product.

This check dequantizes the matrix, measures its mean absolute error, and prints the zero point so we can see how real zero is encoded.

```
print(W_q)
print(W)

#TODO: dequantize W_q and check how close it is to the original W
W_deq = minmax_dequantize(W_q, scale_W, zero_point_W)
print(W_deq)
W_diff = (W - W_deq).abs().mean()
print(W_diff)
print("zero point", zero_point_W)
```

```
tensor([[  41,   19, -128,   86,  -43],
        [ -67,   73,  107,  -15,   10],
        [  -5,   56,  -25,  127,  -42]], dtype=torch.int8)
tensor([[ 0.0000, -0.2934, -2.1788,  0.5684, -1.0845],
        [-1.3986,  0.4033,  0.8380, -0.7193, -0.4033],
        [-0.5966,  0.1820, -0.8567,  1.1006, -1.0712]])
tensor([[-0.0054, -0.2883, -2.1788,  0.5733, -1.0857],
        [-1.3943,  0.4061,  0.8434, -0.7256, -0.4041],
        [-0.5970,  0.1875, -0.8542,  1.1006, -1.0728]])
tensor(0.0031)
zero point tensor(41.4189)
```

The full int8 range is used and the matrix error is small. The nonzero zero point shows that a real zero is represented by an offset code rather than by integer zero.

As in the symmetric case, the min-max integer dot product uses int16 accumulation. The result is kept in code space until the zero-point offsets are removed.

```
#TODO:  perform the matrix multiplication using the quantized values converted to int16
out_q = W_q.to(torch.int16) @ x_q.to(torch.int16)
```

This cell prepares the raw dot products; interpreting them as real values requires the correction terms in the next computation.

Expanding the two affine quantization equations gives the corrected product. The row-wise matrix-code sums, vector-code sum, and shared inner dimension account for the cross terms introduced by both zero points.

```
# Expand (W_q - zero_point_W) @ (x_q - zero_point_x).
# Use the quantized operand sums, rather than the original floats: rounding
# means the latter no longer give the same dequantized product.
out_deq = (
    out_q.to(torch.float32)
    - zero_point_x * W_q.to(torch.float32).sum(dim=1)
    - zero_point_W * x_q.to(torch.float32).sum()
    + W.shape[1] * zero_point_W * zero_point_x
) / (scale_W * scale_x)
```

The resulting expression removes the offsets before dividing by the product of scales. This is why asymmetric quantization cannot simply multiply the raw int8 result by two scales.

We print the corrected min-max result beside the original output and compute their mean absolute difference. This closes the small numerical comparison.

```
print(out_deq)
print(out)
print((out_deq - out).abs().mean())
```

```
tensor([0.4784, 1.3450, 0.2179])
tensor([0.4829, 1.3345, 0.2121])
tensor(0.0069)
```

The result remains close to the floating-point reference. Although this particular error is small, asymmetric mapping trades exact zero representation for tighter use of the available code range.

1.3 Dynamic quantization on a decoder-only model

The small arithmetic example uses toy values and cannot show storage or generation behavior at model scale. The next sequence compares a float32 baseline with two post-training representations using the same 20 prompts, greedy decoding, and 30 new tokens. All timed variants use CPU, because the PyTorch dynamic int8 path and the installed BitsAndBytes CPU backend are being compared on a common device.

The Llama experiment uses a CPU baseline because PyTorch's dynamic int8 Linear quantization is a CPU path. The same CPU device and QNNPACK engine are used for the baseline and quantized timings to keep the comparison aligned.

```
import torch
import os
import time
from transformers import AutoTokenizer, AutoModelForCausalLM

torch.set_num_threads(4)
# PyTorch dynamic quantization is a CPU backend. Keep the FP32 baseline
# on CPU as well so its timing is comparable to the int8 result.
device = torch.device("cpu")
if "qnnpack" in torch.backends.quantized.supported_engines:
    torch.backends.quantized.engine = "qnnpack"
print("Quantization engine:", torch.backends.quantized.engine)
```

The complete progress or warning stream is reproduced in Appendix A.

```
Quantization engine: qnnpack
```

QNNPACK is the selected quantized operator backend. The widget warning is only a notebook display dependency and is preserved in the diagnostics appendix.

The gated Llama checkpoint is accessed through the existing local Hugging Face credential store. The code checks access without printing or embedding a token, then clears the temporary variable.

```
from huggingface_hub import login, get_token

# Use local Hugging Face authentication; never save a token in this notebook.
token = get_token()
if token is None:
    raise RuntimeError("Llama 3.2 requires an approved Hugging Face account. Run ‘hf auth login‘ locally or set HF_TOKEN,
     then rerun.")
login(token=token, add_to_git_credential=False)
del token
print("Hugging Face authentication configured.")
```

```
Hugging Face authentication configured.
```

Authentication succeeds in the finalized run, allowing the exact gated checkpoint to load in the next step. An earlier failed attempt is retained as historical diagnostics only.

This cell loads the public gated Llama 3.2 1B base model in float32 on CPU and loads a left-padding tokenizer. Setting EOS as the pad token lets differently sized prompts share a batch.

```
model_id = "meta-llama/Llama-3.2-1B"
#TODO: load the model and tokenizer
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float32, low_cpu_mem_usage=True).to(device).eval()
tokenizer = AutoTokenizer.from_pretrained(model_id, padding_side="left")

tokenizer.pad_token = tokenizer.eos_token
```

The model and tokenizer become the common baseline for parameter-size measurement, generation, and the two quantized alternatives.

Rather than infer memory from a parameter count, this helper serializes the state dictionary and measures its file size. It establishes the storage baseline before quantization.

```python
def get_model_size(model):
    """Get the size of the model in MB"""
    torch.save(model.state_dict(), "temp.pth")
    size = os.path.getsize("temp.pth") / 1e6  # size in "MB" (technically, it should be 1024**2, but we approximate to 1e6 to get an easier conversion #params <=> MB)
    os.remove("temp.pth")
    return size

print(f"Model size before quantization {(get_model_size(model)):.2f} MB")
```

```
Model size before quantization 4943.31 MB
```

The value reports serialized model-state size in decimal megabytes; it is not peak runtime memory, which also includes activations and framework overhead.

Twenty copies of one short prompt are tokenized together, then the unquantized model greedily generates 30 new tokens. The batch provides a stable workload for the later CPU timing comparison.

```python
text = "The secret of life is"
# Notice we use a batch of 20 sentences -- we will get better results
# on quantized models when processing a batch of inputs
text = [text]*20

#TODO: encode the text using the tokenizer
inputs = tokenizer(text, return_tensors="pt", padding=True).to(device)

tic = time.time()

with torch.no_grad():
    #TODO: generate the output from the model
    baseline_output = model.generate(**inputs, max_new_tokens=30, do_sample=False, pad_token_id=tokenizer.eos_token_id)

elapsed_time = time.time() - tic

#TODO: decode the output
baseline_decoded = tokenizer.batch_decode(baseline_output, skip_special_tokens=True)

print("Baseline model output:", baseline_decoded)
print("\nTime taken for baseline model:", elapsed_time)
```

```
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:590: UserWarning: ‘do_sample‘ is set to ‘False‘. However, ‘temperature‘ is set to ‘0.6‘ -- this flag is only used in sample-based generation modes. You should set ‘do_sample=True‘ or unset ‘temperature‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:595: UserWarning: ‘do_sample‘ is set to ‘False‘. However, ‘top_p‘ is set to ‘0.9‘ -- this flag is only used in sample-based generation modes. You should set ‘do_sample=True‘ or unset ‘top_p‘.
  warnings.warn(
```

```
Baseline model output: ['The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to', 'The secret of life is to be happy, and to be happy you must be yourself. - Ralph Waldo Emerson\nThe secret of life is to be happy, and to']

Time taken for baseline model: 8.564711809158325
```

The decoded batch preserves each prompt followed by its continuation. The elapsed time is specific to this hardware, batch, and software stack.

PyTorch replaces Linear layers with dynamically quantized int8 versions while leaving other modules, including embeddings, in their existing representation. The serialized state size is measured again.

```
quantized_model = torch.quantization.quantize_dynamic(
    model, {torch.nn.Linear}, dtype=torch.qint8
).to('cpu')

# Model size after quantization
print(f"Model size after quantization {(get_model_size(quantized_model)):.2f} MB")
```

```
/var/folders/s1/c1_g_ntd5nb_f0t11858vs640000gn/T/ipykernel_15547/1416564361.py:1: DeprecationWarning: torch.ao.quantization is
     deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use
     torchao eager mode quantize_ API instead
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torch.ao.quantization.quantize_fx.convert_fx,
     please migrate to use torchao pt2e quantization API instead (prepare_pt2e, convert_pt2e)
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
  quantized_model = torch.quantization.quantize_dynamic(
```

```
Model size after quantization 2288.87 MB
```

The size change reflects only modules supported by this transformation and its metadata. It should not be mistaken for every parameter becoming one byte.

Displaying the module tree makes the transformation concrete: it shows which Linear layers are now dynamically quantized and which layers retain their original types.

```
quantized_model
```

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(128256, 2048)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaSdpaAttention(
          (q_proj): DynamicQuantizedLinear(in_features=2048, out_features=2048, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (k_proj): DynamicQuantizedLinear(in_features=2048, out_features=512, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (v_proj): DynamicQuantizedLinear(in_features=2048, out_features=512, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (o_proj): DynamicQuantizedLinear(in_features=2048, out_features=2048, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): LlamaMLP(
          (gate_proj): DynamicQuantizedLinear(in_features=2048, out_features=8192, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (up_proj): DynamicQuantizedLinear(in_features=2048, out_features=8192, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (down_proj): DynamicQuantizedLinear(in_features=8192, out_features=2048, dtype=torch.qint8, qscheme=torch.
     per_tensor_affine)
          (act_fn): SiLU()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): DynamicQuantizedLinear(in_features=2048, out_features=128256, dtype=torch.qint8, qscheme=torch.per_tensor_affine)
)
```

The printed structure is architectural evidence for the selective conversion performed in the previous cell.

The same 20 prompts are generated with the int8 model using greedy decoding and the same 30-token budget. Comparing strings and elapsed time with the baseline isolates the observed effect of this configuration.

```
tic = time.time()

with torch.no_grad():
    #TODO: generate the output from the quantized model
    output = quantized_model.generate(**inputs, max_new_tokens=30, do_sample=False, pad_token_id=tokenizer.eos_token_id)

elapsed_time = time.time() - tic

#TODO: decode the output
output_decoded = tokenizer.batch_decode(output, skip_special_tokens=True)

print("Quantized model output:", output_decoded)
print("\nTime taken for quantized model:", elapsed_time)
```

```
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:590:
     UserWarning: ‘do_sample‘ is set to ‘False‘. However, ‘temperature‘ is set to ‘0.6‘ -- this flag is only used in sample-
     based generation modes. You should set ‘do_sample=True‘ or unset ‘temperature‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:595:
     UserWarning: ‘do_sample‘ is set to ‘False‘. However, ‘top_p‘ is set to ‘0.9‘ -- this flag is only used in sample-based
     generation modes. You should set ‘do_sample=True‘ or unset ‘top_p‘.
  warnings.warn(
[W1026 17:58:49.303102000 qlinear_dynamic.cpp:251] Warning: Currently, qnnpack incorrectly ignores reduce_range when it is set
     to true; this may change in a future release. (function operator())
```

```
Quantized model output: ['The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is', 'The secret of life is
      the most important thing in the world. It is the most important thing in the world. It is the most important thing in
     the world. It is', 'The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is', 'The secret of life is
      the most important thing in the world. It is the most important thing in the world. It is the most important thing in
     the world. It is', 'The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is', 'The secret of life is
      the most important thing in the world. It is the most important thing in the world. It is the most important thing in
     the world. It is', 'The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is', 'The secret of life is
      the most important thing in the world. It is the most important thing in the world. It is the most important thing in
     the world. It is', 'The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is', 'The secret of life is
      the most important thing in the world. It is the most important thing in the world. It is the most important thing in
     the world. It is', 'The secret of life is the most important thing in the world. It is the most important thing in the
     world. It is the most important thing in the world. It is', 'The secret of life is the most important thing in the world.
      It is the most important thing in the world. It is the most important thing in the world. It is']

Time taken for quantized model: 11.583800077438354
```

The full decoded output and runtime are preserved. Any speed or text difference is specific to the tested backend and does not establish a general performance or quality guarantee.

After deleting the float32 and int8 models, this cell loads a fresh 4-bit BitsAndBytes model on CPU. Explicit cleanup avoids holding all large variants together; the same prompt batch is moved to the new model's device.

```
from transformers import BitsAndBytesConfig
import gc

# Release both previous models before loading the next one on this 16 GB Mac.
del model, quantized_model
gc.collect()

# BitsAndBytes 0.50.2 supports the CPU backend on macOS ARM64. Keep all
# three timed experiments on CPU here so their timings are comparable.
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float32,
)
quantized_model = AutoModelForCausalLM.from_pretrained(
    model_id, quantization_config=quantization_config,
    torch_dtype=torch.float32, device_map="cpu", low_cpu_mem_usage=True,
).eval()
inputs = inputs.to(quantized_model.device)
print("4-bit backend: BitsAndBytes (CPU)")
print(f"Model size after quantization: {get_model_size(quantized_model):.2f} MB")
```

```
4-bit backend: BitsAndBytes (CPU)
```

```
Model size after quantization: 1598.46 MB
```

The output confirms the CPU int4 backend and measures the serialized quantized state. The configuration stores 4-bit weights with float32 computation; weight storage precision and arithmetic precision are distinct.

The final generation repeats the same greedy prompt batch and token budget with the int4 model, completing the baseline/int8/int4 comparison.

```
tic = time.time()

```

```
with torch.no_grad():
    #TODO: generate the output from the quantized model
    output = quantized_model.generate(**inputs, max_new_tokens=30, do_sample=False, pad_token_id=tokenizer.eos_token_id)

elapsed_time = time.time() - tic

#TODO: decode the output
output_decoded = tokenizer.batch_decode(output, skip_special_tokens=True)

print("\nquantized model output:", output_decoded)
print("\nTime taken for quantized model:", elapsed_time)
```

```

quantized model output: ['The secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\
    nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret of life
    is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the
     fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The
    secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to
    live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret of life is to live it to the
    fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret
     of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live
    it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', '
    The secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is
     to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret of life is to live it to the
    fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret
     of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live
    it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', '
    The secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is
     to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret of life is to live it to the
    fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret
     of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live
    it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', '
    The secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is
     to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret of life is to live it to the
    fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live it to the fullest.\nThe secret
     of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', 'The secret of life is to live
    it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest', '
    The secret of life is to live it to the fullest.\nThe secret of life is to live it to the fullest.\nThe secret of life is
     to live it to the fullest']

Time taken for quantized model: 16.536229133605957
```

The generated strings and elapsed time are evidence for this run only. Quantization level alone does not predict latency or generation quality independently of the backend and workload.

2 Low-Rank Adaptation: manual adapters and PEFT

The second exercise adapts BERT for binary IMDB sentiment classification without updating all of its weights. The manual implementation adds rank-eight paths to the six Linear transformations in each encoder block, freezes the base model, and trains the adapters on a seeded 2,000-review subset. A second run uses Hugging Face PEFT with the same selected transformations and Trainer configuration. We compare parameter counts and held-out accuracy while keeping visible a key difference: PEFT also trains the newly initialized classification head, whereas the manual path freezes it.

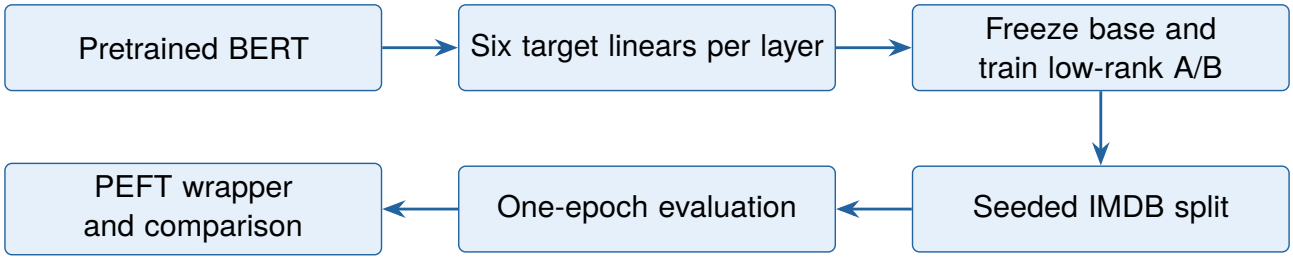


Figure 2: The exercise pipeline, from input to checked result.

2.1 Build and train the manual adapter

The LoRA exercise begins with PyTorch and BERT imports, a fixed seed, and an MPS availability check. This provides deterministic initialization and records the accelerator environment for the later training comparison.

```
import torch
import torch.nn as nn
from transformers import BertTokenizer, BertForSequenceClassification, set_seed

set_seed(42)
torch.set_num_threads(4)
print("MPS available:", torch.backends.mps.is_available())
```

The complete progress or warning stream is reproduced in Appendix A.

```
MPS available: True
```

MPS is available on this host. The notebook's Trainer selects the device during training; the availability message alone does not prove that a particular run used it.

This small reporting helper totals trainable and total parameters and expresses the trainable fraction as a percentage. We use it before and after adapter insertion to quantify the parameter savings.

```
# From https://github.com/huggingface/peft/issues/41#issuecomment-1404611868
def print_trainable_parameters(model):
    """
    Prints the number of trainable parameters in the model.
    """
    trainable_params = 0
    all_param = 0
    for _, param in model.named_parameters():
        all_param += param.numel()
        if param.requires_grad:
            trainable_params += param.numel()
    print(
        f"trainable params: {trainable_params} || all params: {all_param} || trainable%: {100 * trainable_params / all_param:.2f}"
    )
```

The function produces no printed result until called on a model; it is reused for both the manual and PEFT implementations.

We load the uncased BERT encoder with a two-class sequence-classification head and its matching tokenizer. The pretrained encoder supplies language features; the classification head is newly initialized for sentiment labels.

```
model_name = "bert-base-uncased"
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=2)
tokenizer = BertTokenizer.from_pretrained(model_name)
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly
      initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

Transformers warns that the classifier weights are random and must be trained for the task. The head's trainability later becomes an important difference between the manual and PEFT runs.

Before modifying BERT, the helper establishes the full fine-tuning baseline: every parameter in the encoder and classifier is trainable.

```
print_trainable_parameters(model)
```

```
trainable params: 109483778 || all params: 109483778 || trainable%: 100.00
```

All 109,483,778 parameters are initially trainable. This count is compared with the much smaller adapter-only parameter set.

The manual adapter wraps a linear layer with its original operation plus a rank-eight update $(xA)B$. The standard initialization uses Kaiming-uniform A and zero B, so the adapter initially contributes exactly zero while its parameters can learn.

```
class LoRA(nn.Module):
    def __init__(self, original_layer, rank=8):
        super(LoRA, self).__init__()
        self.original_layer = original_layer
        self.rank = rank
        self.in_features = original_layer.in_features
        self.out_features = original_layer.out_features

        # Match the standard LoRA initialization: A uses Kaiming uniform, B starts at zero.
        # Transposing A for initialization gives Kaiming's fan-in the input dimension.
        self.A = nn.Parameter(torch.empty(self.in_features, rank))
        nn.init.kaiming_uniform_(self.A.T, a=5 ** 0.5)
        self.B = nn.Parameter(torch.zeros(rank, self.out_features))

    def forward(self, x):
        # The output is the original layer output plus the low-rank adaptation

        # LoRA output
        lora_output = (x @ self.A) @ self.B

        # layer output, which combines the original output with the LoRA one
        return self.original_layer(x) + lora_output
```

This class preserves the base projection and adds a low-rank trainable path. Its parameter dimensions are input-by-rank and rank-by-output, so the added count is proportional to rank rather than input times output.

Each BERT encoder block has six linear projections selected here: query, key, value, attention output, and the two feed-forward transformations. Replacing them applies the same adapter construction throughout the encoder.

```
for layer in model.bert.encoder.layer:
    layer.attention.self.query = LoRA(layer.attention.self.query)
    layer.attention.self.key = LoRA(layer.attention.self.key)
    layer.attention.self.value = LoRA(layer.attention.self.value)
    layer.attention.output.dense = LoRA(layer.attention.output.dense)
    layer.intermediate.dense = LoRA(layer.intermediate.dense)
    layer.output.dense = LoRA(layer.output.dense)
```

The model now contains a base projection plus a low-rank path at each selected site. Layer normalization, embeddings, and the task classifier are not replaced by this loop.

To make the experiment parameter-efficient, all model parameters are frozen first. The loop then enables gradients only for each manually inserted A and B matrix.

```
# Freeze all parameters except the LoRA parameters

# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False

# Unfreeze only LoRA parameters
for layer in model.modules():
    if isinstance(layer, LoRA):
        layer.A.requires_grad = True
        layer.B.requires_grad = True
```

This policy trains the adapters while leaving the pretrained encoder and the randomly initialized classifier frozen. That frozen classifier is a limitation when comparing accuracy with PEFT, which also trains its task head.

The parameter helper now checks whether freezing and replacement produced the expected trainable count. The assertion guards the architecture calculation against a missed or duplicated projection.

```
print_trainable_parameters(model)
assert sum(p.numel() for p in model.parameters() if p.requires_grad) == 1_327_104
```

```
trainable params: 1327104 || all params: 110810882 || trainable%: 1.20
```

The six adapter sites across twelve layers contain 1,327,104 trainable parameters, about 1.20

The IMDB dataset is shuffled with seed 42 and reduced to 2,000 training and 1,000 test examples. This small, disjoint sample keeps the demonstration manageable while preserving a reproducible comparison set.

```
from datasets import load_dataset

# Load a sentiment analysis dataset
dataset = load_dataset('imdb')
train_dataset = dataset['train'].shuffle(seed=42).select(range(2000))
test_dataset = dataset['test'].shuffle(seed=42).select(range(1000))
```

```
Using the latest cached version of the dataset since imdb couldn't be found on the Hugging Face Hub (offline mode is enabled).
```

```
Found the latest cached dataset configuration 'plain_text' at /Users/nicolabeccaceci/.cache/huggingface/datasets/imdb/
    plain_text/0.0.0/e6281661ce1c48d982bc483cf8a173c1bbeb5d31 (last modified on Mon Oct 26 09:43:53 2026).
```

The cached dataset is used in offline mode. The same selected rows feed both adapter implementations.

This metric callback converts the Trainer's logits to class predictions and computes accuracy against the reference labels. It is reused for initial evaluation, periodic validation, and final comparison.

```
from sklearn.metrics import accuracy_score

# Function to compute accuracy
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    accuracy = accuracy_score(labels, preds)
    return {"accuracy": accuracy}
```

The callback returns a single accuracy metric for each evaluation event, keeping the comparison focused on held-out classification accuracy.

The tokenizer converts the review text into padded, truncated BERT inputs. Mapping both splits in batches prepares model-ready data once for both training approaches.

```
# Tokenize the dataset
def tokenize_function(sample):
    return tokenizer(sample['text'], padding="max_length", truncation=True)

train_dataset = train_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)
```

The tokenized datasets contain aligned input IDs, attention masks, and labels; later Trainers can reuse them without retokenizing.

The Trainer configuration keeps an effective batch of 32 using microbatches of four and eight accumulation steps. It specifies one epoch, the shared learning rate and weight decay, evaluation and logging every ten updates, and disables external reporting.

```
from transformers import Trainer, TrainingArguments

batch_size = 32
num_train_epochs = 1

learning_rate = 2e-4
weight_decay = 0.01

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    eval_strategy="steps",
    eval_steps=10,
    learning_rate=learning_rate,
    # Accumulate smaller microbatches to retain effective batch size 32.
```

```
    per_device_train_batch_size=4,
    gradient_accumulation_steps=batch_size // 4,
    per_device_eval_batch_size=8,
    num_train_epochs=num_train_epochs,
    weight_decay=weight_decay,
    logging_dir='./logs',  # Directory for storing logs
    logging_steps=10,  # Log every 10 steps
    report_to="none",
    seed=42,
    dataloader_pin_memory=torch.cuda.is_available(),
)

# Initialize the Trainer object
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

The same arguments and metric callback are reused for both implementations. This cell creates the training controller but does not yet evaluate or update the model.

Before training, we measure the manual adapter model on the held-out split. This establishes the starting accuracy of the frozen random classifier plus untrained adapters.

```
# Evaluate the model
results = trainer.evaluate()
print(f"Accuracy on the validation set: {results['eval_accuracy']:.4f}")
```

The complete progress or warning stream is reproduced in Appendix A.

```
Accuracy on the validation set: 0.5070
```

Initial accuracy is 0.5070, close to chance for two balanced classes. The progress trace is retained in the diagnostics appendix.

The manual adapters are trained for one epoch on the seeded 2,000-review subset, with held-out evaluation at regular update intervals. The complete validation table shows how accuracy and loss change during adaptation.

```
results = trainer.train()
```

```
Progress: 62/62
[62/62 13:22, Epoch 0/1]
Step
Training Loss
Validation Loss
Model Preparation Time
Accuracy
10
0.698200
0.686084
0.002800
0.582000
20
0.694700
0.681265
0.002800
0.620000
30
0.683300
0.675302
0.002800
0.643000
40
0.686100
0.671291
0.002800
0.661000
50
0.671200
0.667917
0.002800
0.654000
60
0.670200
0.666133
0.002800
0.657000
```

The 62-update run reports the per-checkpoint losses and accuracies in full. Its training time is about 810 seconds on this host; the assigned classifier remains frozen, so interpret this result as the notebook's adapter-only implementation rather than a fully matched classifier fine-tune.

After the manual adapter epoch, the same held-out set is evaluated again to quantify its endpoint under the shared metric.

```
# Evaluate the model
results = trainer.evaluate()
print(f"Accuracy on the validation set: {results['eval_accuracy']:.4f}")
```

The complete progress or warning stream is reproduced in Appendix A.

```
Accuracy on the validation set: 0.6580
```

Accuracy rises from 0.5070 to 0.6580. This is an observed result on 1,000 selected reviews and the fixed random classifier, not a general benchmark.

2.2 Apply the same encoder adapters with PEFT

The PEFT path starts from a freshly loaded BERT checkpoint after the manual model is released. Its target-module list is derived from the same six encoder linears per layer. The library also keeps the sequence-classification head trainable; this adds 1,538 parameters and means the final accuracy comparison is informative but not perfectly controlled.

Before testing PEFT, the notebook releases the manual Trainer and model, clears cached MPS allocations, resets the seed, and reloads a fresh BERT classifier. This prevents learned weights from leaking into the second approach.

```
# Release the manually adapted model/trainer before loading a fresh baseline.
import gc
globals().pop("trainer", None)
globals().pop("model", None)
gc.collect()
if torch.backends.mps.is_available():
    torch.mps.empty_cache()

set_seed(42)
model_name = "bert-base-uncased"
model = BertForSequenceClassification.from_pretrained(model_name)
tokenizer = BertTokenizer.from_pretrained(model_name)
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly
      initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

The warning again identifies the newly initialized classification head. Reloading it under the same seed makes the PEFT branch start from a reproducible baseline.

PEFT's LoRA configuration uses rank eight, alpha 32, dropout 0.1, and targets the same six encoder Linear projections per layer. Its sequence-classification wrapper also keeps the task head trainable.

```
from peft import LoraConfig, get_peft_model, PeftModelForSequenceClassification

config = LoraConfig(
    r=8,
    lora_alpha=32,
    lora_dropout=0.1,
    bias="none",
    # Select the same six linear transformations in each encoder block.
    target_modules=[
        name for name, module in model.named_modules()
        if name.startswith("bert.encoder.layer.") and isinstance(module, nn.Linear)
    ],
)

peft_model = PeftModelForSequenceClassification(model, peft_config=config)
```

The PEFT model now contains low-rank modules at the selected sites. Its trainable count will include the classification head as well as the adapters.

This type inspection confirms that `get_peft_model` returned the sequence-classification PEFT wrapper expected for the task.

```
type(peft_model)
```

```
peft.peft_model.PeftModelForSequenceClassification
```

The Python type is 'PeftModelForSequenceClassification', so the task-specific classifier is preserved by the wrapper.

Printing the first encoder block verifies where PEFT inserted its A and B projections and shows that the original linear layers remain as the adapter base.

```
peft_model.bert.encoder.layer[0]
```

```
BertLayer(
  (attention): BertAttention(
    (self): BertSdpaSelfAttention(
      (query): lora.Linear(
        (base_layer): Linear(in_features=768, out_features=768, bias=True)
        (lora_dropout): ModuleDict(
          (default): Dropout(p=0.1, inplace=False)
        )
        (lora_A): ModuleDict(
          (default): Linear(in_features=768, out_features=8, bias=False)
        )
        (lora_B): ModuleDict(
          (default): Linear(in_features=8, out_features=768, bias=False)
        )
        (lora_embedding_A): ParameterDict()
        (lora_embedding_B): ParameterDict()
        (lora_magnitude_vector): ModuleDict()
      )
      (key): lora.Linear(
        (base_layer): Linear(in_features=768, out_features=768, bias=True)
        (lora_dropout): ModuleDict(
          (default): Dropout(p=0.1, inplace=False)
        )
        (lora_A): ModuleDict(
          (default): Linear(in_features=768, out_features=8, bias=False)
        )
        (lora_B): ModuleDict(
          (default): Linear(in_features=8, out_features=768, bias=False)
        )
        (lora_embedding_A): ParameterDict()
        (lora_embedding_B): ParameterDict()
        (lora_magnitude_vector): ModuleDict()
      )
      (value): lora.Linear(
        (base_layer): Linear(in_features=768, out_features=768, bias=True)
        (lora_dropout): ModuleDict(
          (default): Dropout(p=0.1, inplace=False)
        )
        (lora_A): ModuleDict(
          (default): Linear(in_features=768, out_features=8, bias=False)
        )
        (lora_B): ModuleDict(
          (default): Linear(in_features=8, out_features=768, bias=False)
        )
        (lora_embedding_A): ParameterDict()
        (lora_embedding_B): ParameterDict()
        (lora_magnitude_vector): ModuleDict()
      )
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): BertSelfOutput(
      (dense): lora.Linear(
        (base_layer): Linear(in_features=768, out_features=768, bias=True)
        (lora_dropout): ModuleDict(
          (default): Dropout(p=0.1, inplace=False)
        )
        (lora_A): ModuleDict(
          (default): Linear(in_features=768, out_features=8, bias=False)
        )
        (lora_B): ModuleDict(
          (default): Linear(in_features=8, out_features=768, bias=False)
        )
        (lora_embedding_A): ParameterDict()
        (lora_embedding_B): ParameterDict()
        (lora_magnitude_vector): ModuleDict()
      )
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate): BertIntermediate(
    (dense): lora.Linear(
      (base_layer): Linear(in_features=768, out_features=3072, bias=True)
      (lora_dropout): ModuleDict(
```

```
        (default): Dropout(p=0.1, inplace=False)
      )
      (lora_A): ModuleDict(
        (default): Linear(in_features=768, out_features=8, bias=False)
      )
      (lora_B): ModuleDict(
        (default): Linear(in_features=8, out_features=3072, bias=False)
      )
      (lora_embedding_A): ParameterDict()
      (lora_embedding_B): ParameterDict()
      (lora_magnitude_vector): ModuleDict()
    )
    (intermediate_act_fn): GELUActivation()
  )
  (output): BertOutput(
    (dense): lora.Linear(
      (base_layer): Linear(in_features=3072, out_features=768, bias=True)
      (lora_dropout): ModuleDict(
        (default): Dropout(p=0.1, inplace=False)
      )
      (lora_A): ModuleDict(
        (default): Linear(in_features=3072, out_features=8, bias=False)
      )
      (lora_B): ModuleDict(
        (default): Linear(in_features=8, out_features=768, bias=False)
      )
      (lora_embedding_A): ParameterDict()
      (lora_embedding_B): ParameterDict()
      (lora_magnitude_vector): ModuleDict()
    )
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
```

The module tree displays adapters on the attention and feed-forward linears. This is the library-managed counterpart to the manual replacements above.

The PEFT helper calculates trainable parameters after wrapping. Comparing the result with the manual count reveals whether the task head is included.

```
peft_model.print_trainable_parameters()
```

```
trainable params: 1,328,642 || all params: 110,812,420 || trainable%: 1.1990
```

PEFT trains 1,328,642 parameters, including 1,538 classifier weights and biases, versus 1,327,104 manual adapter parameters. The accuracy comparison therefore is not perfectly controlled.

A second Trainer is constructed with the same tokenized splits, metric callback, and training arguments. Only the model object changes to the PEFT-wrapped classifier.

```
from transformers import Trainer, TrainingArguments

# Initialize the Trainer object
trainer = Trainer(
    model=peft_model,
    args=training_args, # we will recycle the same training arguments as before!
    train_dataset=train_dataset, # also datasets,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics # and compute_metrics
)
```

The comparison is ready to run with the same one-epoch schedule and effective batch size as the manual path.

PEFT adapters are trained for one epoch using the shared Trainer configuration. The saved table records validation loss and accuracy at each logged checkpoint.

```
trainer.train()
```

```
Progress: 62/62
[62/62 13:26, Epoch 0/1]
Step
Training Loss
Validation Loss
Accuracy
10
0.694100
0.673932
0.641000
```

```
20
0.673600
0.642261
0.686000
30
0.619900
0.574219
0.714000
40
0.574400
0.481102
0.781000
50
0.461800
0.449831
0.789000
60
0.445800
0.416675
0.817000
```

```
TrainOutput(global_step=62, training_loss=0.5760850579507889, metrics={'train_runtime': 814.3872, 'train_samples_per_second':
    2.456, 'train_steps_per_second': 0.076, 'total_flos': 530110204870656.0, 'train_loss': 0.5760850579507889, 'epoch':
    0.992})
```

The 62-update run reports training loss 0.5761 and runtime 814.4 seconds; the full step table is preserved. The task head is trainable here, unlike in the manual branch, so the accuracy difference combines implementation and head-training effects.

The final cell evaluates the trained PEFT classifier once more on the held-out reviews so its endpoint can be read alongside the manual result.

```
# Evaluate the model
results = trainer.evaluate()
print(f"Accuracy on the validation set: {results['eval_accuracy']:.4f}")
```

The complete progress or warning stream is reproduced in Appendix A.

```
Accuracy on the validation set: 0.8170
```

Final accuracy is 0.8170 on this selected test subset. The difference from the manual 0.6580 result is suggestive for this run, but the two branches differ in classifier trainability and PEFT dropout.

A Diagnostic output and earlier authentication attempt

The complete warnings and progress-only streams omitted from the computational narrative are retained here. They remain tied to their source outputs by the coverage manifest and raw payload hashes.

Diagnostic stream from text-01-quantization.ipynb, cell 39.

```
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please
      update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

Diagnostic stream from text-02-LoRA.ipynb, cell 3.

```
/Users/nicolabeccaceci/.venvs/llms-lab05/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please
      update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

Diagnostic stream from text-02-LoRA.ipynb, cell 22.

```
Progress: 250/125
[125/125 03:28]
```

Diagnostic stream from text-02-LoRA.ipynb, cell 24.

```
Progress: 125/125
[125/125 01:09]
```

Diagnostic stream from text-02-LoRA.ipynb, cell 38.

```
Progress: 125/125
[125/125 01:06]
```

Earlier authentication failure. The first saved attempt stopped at the notebook's local Hugging Face authentication check. The later execution used the account's existing credential store and successfully loaded the approved checkpoint. No credential value is included here.

```

RuntimeError: Llama 3.2 requires an approved Hugging Face account. Run hf auth login locally or set HF_TOKEN, then rerun.
```

B Execution and reproducibility notes

No Lab05 lecture recording was available among the supplied course files. The only additional Whisper item inspected was a computer-vision lecture on object detection and segmentation, so it was excluded as unrelated. The professor's promised Laboratory 05 recording can be incorporated later. Explanations and numerical claims here follow the notebook source and its recorded executions rather than inferred lecture content.

The quantization arithmetic uses seeded toy tensors. The Llama comparison uses checkpoint revisions and exact package versions recorded in `lab05/execution/environment.json`; CPU timing depends on the host, thread scheduling, backend, and model-cache state. Serialized state size is not peak inference memory. The LoRA runs use a seeded 2,000/1,000 IMDB subset, microbatch four, eight-step accumulation, and one epoch. Manual training took about 810 seconds and PEFT about 814 seconds on this host. The manual path freezes the new classification head, while PEFT trains it, so the 0.658 versus 0.817 accuracy results are not an isolated causal comparison of the adapter implementations.